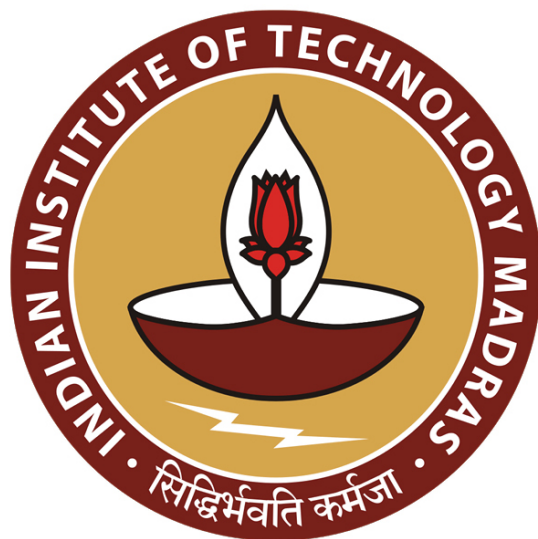


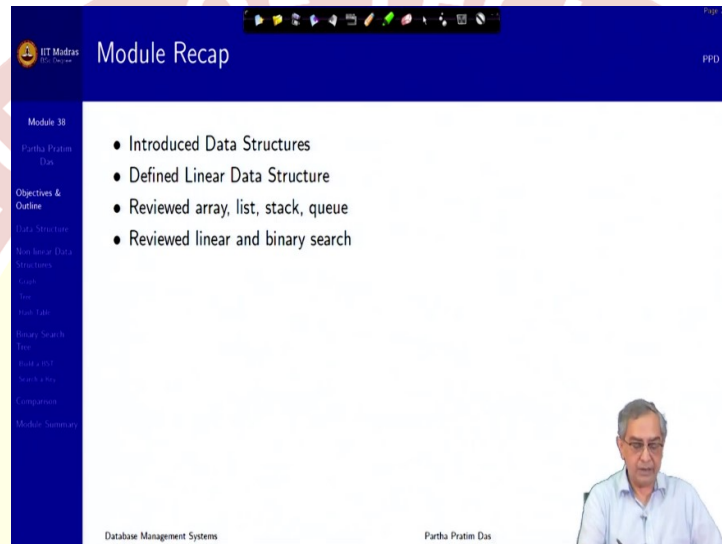


IIT Madras
ONLINE DEGREE

Database Management Systems
Professor. Partha Pratim Das
Department of Computer Science & Engineering
Indian Institute of Technology, Kharagpur
Algorithms and Data Structures/3: Data Structures

Welcome to Module 38 of Database Management Systems course, in the IIT Madras online BSc program.

(Refer Slide Time: 0:27)



The screenshot shows a presentation slide titled "Module Recap" from the IIT Madras online BSc program. The slide is part of a presentation titled "Database Management Systems". The slide content includes a list of topics covered in the module:

- Introduced Data Structures
- Defined Linear Data Structure
- Reviewed array, list, stack, queue
- Reviewed linear and binary search

A small video inset in the bottom right corner shows Professor Partha Pratim Das. The slide is part of a presentation titled "Database Management Systems".

We have been discussing about simple data structures in order to prepare ourselves for how the storage and data management in the physical layer would be done. In the previous module, we have talked about linear data structures, the common ones that we are always familiar with array, list and built on them are stack, queue, etc. We have also looked at what are the different options of searching a key in such data structures.

(Refer Slide Time: 1:03)

Module Objectives

- Introducing Non-linear Data Structures - graph, tree, hash table
- Exploring Binary Search Tree
- Comparing Linear and Non-Linear Data Structures

Database Management Systems Partha Pratim Das

Now, in the module, in this module, we will go onto nonlinear data structures, graph, tree, hash table and so on, and we will specifically explore binary search trees. And at the end, we will compare the linear and non-linear data structures. So, these are the three main topics, as you see on the left.

(Refer Slide Time: 1:29)

Data Structure

- **Data structure:** A data structure specifies the way of organizing and storing **in-memory** data that enables **efficient access** and **modification** of the data.
 - Linear Data Structures
 - Non-linear Data Structures
- Most data structure has a container for the data and typical operations that its needs to perform
- For applications relating to data management, the key operations are:
 - Create
 - Insert
 - Delete
 - Find / Search
 - Close
- Efficiency is measured in terms of time and space taken for these operations

Database Management Systems Partha Pratim Das 38.5

So, this is just to quickly recap a data structure specifies a way of organizing and storing in-memory data, emphasize that, and it enables efficient access and modification of the data. An efficiency is measured as a combination of time, which is primarily the access but we also consider the efficiency of space.

We have talked about non-linear, linear data structure, it is time for the non-linear data structure. Remember that all data structure at least most of them have a container for the data, which they are going to structure and manage and a typical operation that needs to be done and it is with respect to these operations that we want to measure the time efficiency and the space. An operations typically include create, insert, delete, find, access, close and so on.

(Refer Slide Time: 2:27)

Non-linear Data Structures

Non-linear Data Structures: Why?

- From the study of **Linear data structures** in the last module, we can make the following summary observations:
 - All of them have the space complexity $O(n)$, which optimal. However, the actual used space may be lower in array while linked list has an overhead of 100% (double)
 - All of them have complexities that are identical for Worst as well as Average case
 - All of them offer satisfactory complexity for some operations while being unsatisfactory on the others

	Array		Linked List	
	Unordered	Ordered	Unordered	Ordered
✓ Access	$O(1)$	$O(1)$	$O(n)$	$O(n)$
✓ Insert	$O(n)$	$O(n)$	$O(1)$	$O(1)$
✓ Delete	$O(n)$	$O(n)$	$O(1)$	$O(1)$
✓ Search	$O(n)$	$O(\lg n)$	$O(n)$	$O(n)$

- Non-Linear data structures** can be used to trade-off between extremes and achieve a balanced good performance for all

Now, coming to non-linear data structure, let us recall that from a study of linear data structure that all of them that we have seen in that array and linked list, ordered and unordered all of them have the same space complexity which is order n , and which kind of is optimal. However, we can always note, we should always note that the actual space used may

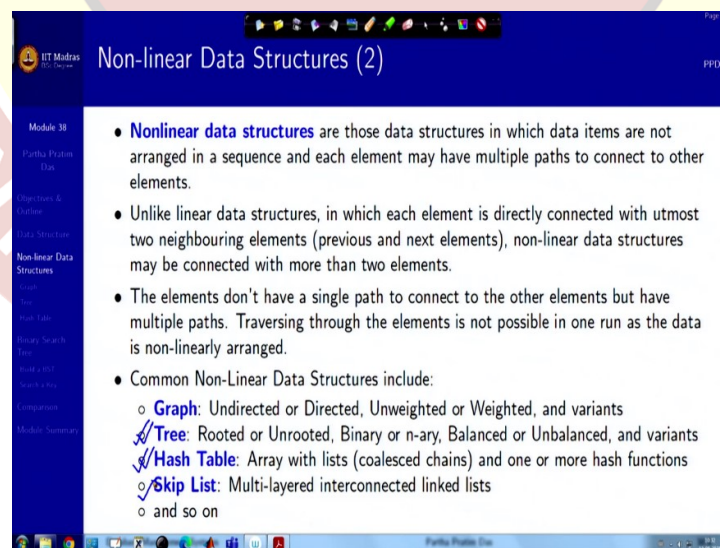
be lower in an array, because there may be unused space while in linked list there is an overhead of 100% because with every data node you will have to keep a, keep a pointer.

I mean, we cannot, I mean in the order it is 100% in reality, it may not be so because the data node may be much larger than the pointer itself. So, all of them also have complexities which are identical. This is incidental, this is just an observation. Incidentally, all of them have respective complexities of operations which are identical between the worst case and the average case, so we can just talk about one type of case.

And all of them offer satisfactory complexity may be very good complexity for some operations while they are very unsatisfactory for others. So, this is a summarization table for it. So, looking at the four primary operations that we would typically need in a database application, we can see that in terms of an array the access is good, while in terms of a linked list, it is not. Insert, delete is really bad for arrays, and they are excellent for linked list.

And search is I mean, kind of bad for everyone, except if I have an ordered array where you can do binary search. So, it clearly says that neither I can use array efficiently for all operations nor I can use linked list, so, we have to go to a non-linear structure where we trade-off between these extreme and kind of workout a moderate good for each and every operation.

(Refer Slide Time: 4:38)



The screenshot shows a presentation slide titled "Non-linear Data Structures (2)" from IIT Madras. The slide content is as follows:

- **Nonlinear data structures** are those data structures in which data items are not arranged in a sequence and each element may have multiple paths to connect to other elements.
- Unlike linear data structures, in which each element is directly connected with utmost two neighbouring elements (previous and next elements), non-linear data structures may be connected with more than two elements.
- The elements don't have a single path to connect to the other elements but have multiple paths. Traversing through the elements is not possible in one run as the data is non-linearly arranged.
- Common Non-Linear Data Structures include:
 - **Graph**: Undirected or Directed, Unweighted or Weighted, and variants
 - ✓ **Tree**: Rooted or Unrooted, Binary or n-ary, Balanced or Unbalanced, and variants
 - ✓ **Hash Table**: Array with lists (coalesced chains) and one or more hash functions
 - ✓ **Skip List**: Multi-layered interconnected linked lists
 - and so on

A nonlinear data structure, as the name suggests, it does not organize the data in a linear sequence rather, it is kind of a network structure where the data elements are kind of organized through their linkages and those linkages may not be as simple as a linked list like

previous and next elements kind of, but they could be different kinds of hierarchical linkages that may be possible, and there may be more than one path to reach one element from another element and so on.

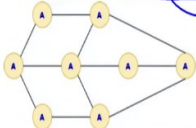
There are several nonlinear data structures, but we will, I will mention about the few major ones. One is graph, which could be undirected, directed and so on, we will talk a little bit more about that. The tree, which is going to be a mainstay for us in this discussion in the database applications, which could be varied kinds as well.

Hash tables, which is also heavily used in database applications, and we will dedicate a separate module for discussing that. And some other ones like skip list, which we will not discuss here, but it is very interesting that it has multiple linked list in a layered interconnected fashion and gives you very interesting performance measures.

(Refer Slide Time: 5:59)

Non-linear Data Structures (3): Graph

- **Graphs:** Graph G is a collection of vertices V (store the elements) and connecting edges (links) E between vertices: $G = \langle V, E \rangle$ where $E \subseteq V \times V$



- A graph may be:
 - Undirected or Directed ✓
 - Unweighted or Weighted ✓
 - Cyclic or Acyclic ✓
 - Disconnected or Connected ✓
 - and so on
- Examples of a graph include:
 - ER Diagram ✓
 - Network: Electrical, Water ✓
 - Friendships in Facebook
 - Knowledge Graph

Now, coming to the specifics. A graph is basically a collection of nodes, and edges between them. So, mathematically graph is a doublet, which has a set of vertices and a set of edges, so that the set of edges are pairwise tuples of the vertices and there is a example of a undirected unweighted graph.

Now, a graph could be undirected or directed that is the order of the two vertices in the tuple may not or may matter, it could be unweighted or weighted. If it is unweighted, all weights are considered to be same and 1. It could have cycle or it may not have cycle, it may be connected or disconnected and so on several properties, and graph occur very predominantly everywhere.

For example, we have seen a major application of graph in terms of the ER diagram. Every ER diagram is actually a graph, and they are interesting, they actually have more than one type of node, nodes also have different types. But simpler graphs could be different networks like electrical, water, sanitation, these kind of networks.

Even there could be conceptual graphs like friendships and Facebook, the Knowledge Graph and so on, so forth. So, that is a it is a wide variety of applications to which the graph as a nonlinear structure can cater to.

(Refer Slide Time: 7:19)

Non-linear Data Structures (4): Tree

- **Tree:** Is a connected acyclic graph representing hierarchical relationship

A tree may be:

- Rooted or Unrooted
- Binary or n-ary
- Balanced or Unbalanced
- Disconnected (forest) or Connected
- and so on

Examples of a tree include:

- Composite Attributes
- Family Genealogy
- Search Trees
- and so on

Tree on the other hand is very handy. It is a special type of graph, which is connected, there are two major factors in this. One is, it is connected and then it is acyclic, there is no cycle, it is connected. Every node should be reachable from the other, but it does not have a cycle. So, it kind of represents a hierarchical relationship particularly if we have a root for it. It is not necessary for a tree to have a root. I mean, I can have a tree like this.

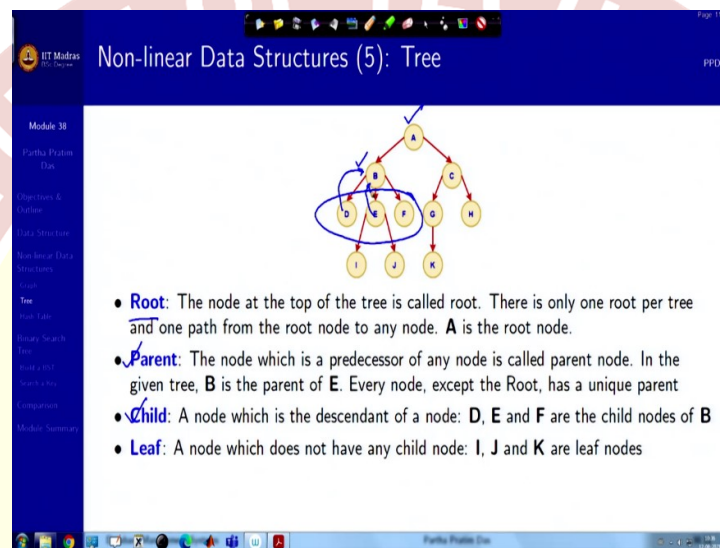
This is a tree, but I have not identified what is the root. So, identifying root is an additional property, but usually we do that so it could be rooted or unrooted. It could be binary or n-ary, that depends on how many neighbors, what is a bound on the number of neighbors a particular vertex may have.

If that bound is 3, 2 child and 1 parent then we say it is a binary node, binary tree, otherwise, it could be n-ary tree for anything. It could be balanced or unbalanced, we will have to look into this property a little bit more. And I said that by definition trees are connected, but often times we also deal with forest which is a collection of trees, which are not connected between

themselves. So, it could be, I mean that kind of a collection of trees is called a forest, we will often need a forest as well.

Now, for example, when we represented composite attributes name, composed of first name, middle name, last name, then we were talking about a tree. When we talked about date, time, a date time is a day a month a year. So, these are different examples of tree. The family tree we all know of the genealogy search trees and so on, we will see a lot more of this.

(Refer Slide Time: 9:15)



Now, since we will use tree heavily, I would like to introduce some definitions, which I am sure, from your algorithms course, data structure course you would know, but just a quick recap. The root is a node, which is identified not to have a parent, and every, we will assume at least in our discussion that there is a unique root for every tree we deal with. The parent of a node is a predecessor in the tree starting from the root, and every node has a unique parent.

The child are the nodes that follow a particular node, so for B, these are the children and for D this is the parent, for E this is a parent. So, the child could be many. So, from a node to the children, it is a one to many relationship, but a parent would always be unique. And a node, which does not have any child is known known as a leaf node. So, simple definitions you all would know.

(Refer Slide Time: 10:16)

Page 13:11

Non-linear Data Structures (6): Tree

Module 38
Parthiv Prabhu Das

Objectives & Outcomes
Unit 38 Structure
Non-linear Data Structures
Tree
Tree Traversal
Binary Search Tree
Heap & AVL
B-Trees
Comprehensions
Module Summary

- **Internal Nodes:** The node which has at least one child is called internal Node
- **Subtree:** Subtree represents the tree rooted at that node
- **Path:** Path refers to the sequence of nodes along the edges of a tree
- **Siblings:** Nodes having the same parents: D, E and F are the siblings.
- **Arity:** Number of children of a node. B has arity 3, E has arity 2, G has arity 1, and D has arity 0 (**Leaf**)

Maximum arity of a node is defined as the arity of the tree.

Parthiv Prabhu Das

Page 13:11

Non-linear Data Structures (6): Tree

Module 38
Parthiv Prabhu Das

Objectives & Outcomes
Unit 38 Structure
Non-linear Data Structures
Tree
Tree Traversal
Binary Search Tree
Heap & AVL
B-Trees
Comprehensions
Module Summary

- **Internal Nodes:** The node which has at least one child is called internal Node
- **Subtree:** Subtree represents the tree rooted at that node
- **Path:** Path refers to the sequence of nodes along the edges of a tree
- **Siblings:** Nodes having the same parents: D, E and F are the siblings.
- **Arity:** Number of children of a node. B has arity 3, E has arity 2, G has arity 1, and D has arity 0 (**Leaf**)

Maximum arity of a node is defined as the arity of the tree.

Parthiv Prabhu Das

Page 13:11

Non-linear Data Structures (6): Tree

Module 38
Parthiv Prabhu Das

Objectives & Outcomes
Unit 38 Structure
Non-linear Data Structures
Tree
Tree Traversal
Binary Search Tree
Heap & AVL
B-Trees
Comprehensions
Module Summary

- **Internal Nodes:** The node which has at least one child is called internal Node
- **Subtree:** Subtree represents the tree rooted at that node
- **Path:** Path refers to the sequence of nodes along the edges of a tree
- **Siblings:** Nodes having the same parents: D, E and F are the siblings.
- **Arity:** Number of children of a node. B has arity 3, E has arity 2, G has arity 1, and D has arity 0 (**Leaf**)

Maximum arity of a node is defined as the arity of the tree.

Parthiv Prabhu Das

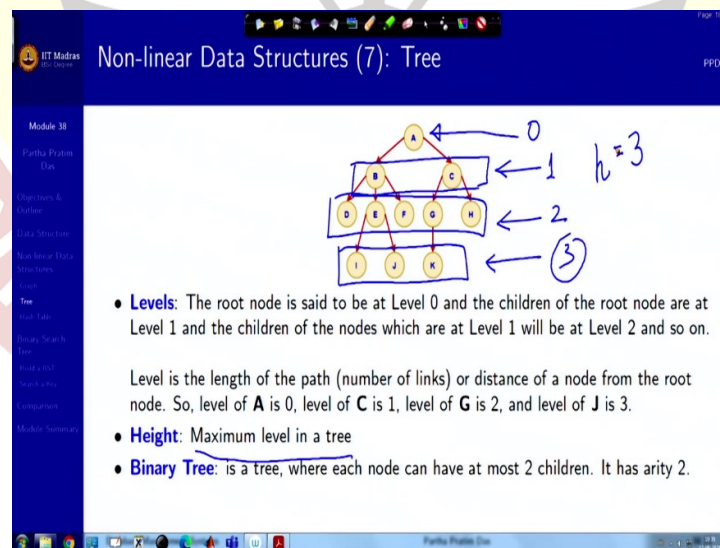
Then anything, which is not a leaf node will be called an internal node. So, this is an internal node, this is an internal node, this is an internal node, but these are not internal nodes because these are leaf nodes, so internal node will have at least one child.

Then you say a sub-tree is one, which is the tree rooted at some node which is not the root. So, this is a sub-tree of rooted at B, this is a subtree rooted at G, this is a subtree rooted at H which has only one node, so that is a concept of a sub-tree, the entire thing hanging from that point.

A path is a sequence of nodes following the edges. Typically, paths will be from root to some node, but it could be that I could also talk about a path which is I, E, B, A, G, C, G this could, this also actually is a path. Siblings are nodes, which have the common parent and arity is the number of children of a node. So, B has an arity 3, C has an arity 2 and so, on. And a leaf node can be defined differently as saying that it has arity 0.

And a maximum arity of a node that exists in the tree is known as the arity of the tree. So, you can say that, for this tree the arity of the tree is 3, because this is the B which has three children it has arity 3 so, this is a ternary tree. And it is not necessary for a n-ary tree that all nodes must have n children.

(Refer Slide Time: 11:59)



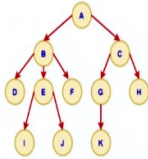
Then you talk about level by identifying that the root is the root so it is at level 0, then the children of the root are considered, are said to be in level 1. The grandchildren take up level 2, great grandchildren take up level 3 and so on. So, level by level you could define a tree, and that becomes very important in terms of several analysis that we need to do. So, we say

the height of a tree is the maximum level in a tree. So, here the maximum level is 3, so we will say height is 3.

In other words, the height, if I take a, start from the root and start going down till I reach a leaf node, I will or till I reach a node, the number of links I cover is the level of that node. So, when I go from A to F, I go 1, I go 2, so F has a level which is 2. So, in this way, you can also define that the height is the, that, those leaf nodes, which has a longest path from the root to that leaf node. Typically, we will deal with binary trees, which have two, maximum of two children at a node.

(Refer Slide Time: 13:37)

Non-linear Data Structures (8): Tree

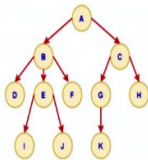


- **Fact 1:** A tree with n nodes has $n - 1$ edges
- **Fact 2:** The maximum number of nodes at level l of a binary tree is 2^l .
- **Fact 3:** If h is the height of a binary tree of n nodes, then:
 - $h + 1 \leq n \leq 2^{h+1} - 1$
 - $\lceil \lg(n+1) \rceil - 1 \leq h \leq n - 1$
 - $O(\lg n) \leq h \leq O(n)$
 - For a k -ary tree, $O(\lg_k n) \leq h \leq O(n)$

Handwritten notes on the right side of the slide:

$$\begin{array}{l} 0 \rightarrow 1-2^0 \\ 1 \rightarrow 2-2^1 \\ 2 \rightarrow 4-2^2 \\ \vdots \rightarrow 2^k \end{array}$$

Non-linear Data Structures (8): Tree



- **Fact 1:** A tree with n nodes has $n - 1$ edges
- **Fact 2:** The maximum number of nodes at level l of a binary tree is 2^l .
- **Fact 3:** If h is the height of a binary tree of n nodes, then:
 - $h + 1 \leq n \leq 2^{h+1} - 1$
 - $\lceil \lg(n+1) \rceil - 1 \leq h \leq n - 1$
 - $O(\lg n) \leq h \leq O(n)$
 - For a k -ary tree, $O(\lg_k n) \leq h \leq O(n)$

Handwritten notes on the right side of the slide:

$$\begin{array}{l} \text{A vertical chain of nodes} \\ n \leq 1 + 2 + 2^2 + \dots + 2^h \\ = 2^{h+1} - 1 \end{array}$$

Non-linear Data Structures (8): Tree

- **Fact 1:** A tree with n nodes has $n - 1$ edges
- **Fact 2:** The maximum number of nodes at level l of a binary tree is 2^l .
- **Fact 3:** If h is the height of a binary tree of n nodes, then:
 - $h + 1 \leq n \leq 2^{h+1} - 1$ ✓
 - $\lceil \lg(n+1) \rceil - 1 \leq h \leq n - 1$ ✓
 - $O(\lg n) \leq h \leq O(n)$ ✓
 - For a k -ary tree, $O(\lg_k n) \leq h \leq O(n)$ ✓

Handwritten note: $O(\lg n) \leq h \leq O(n)$

Now, there are few facts, which I present without direct proof. If you do not know, you can always look up workout or always look up the book. One is a tree with n nodes have exactly $n-1$. It does not matter what is the arity, it does not matter whether it is rooted or unrooted it will have to have $n-1$ edges, you can try to prove that.

Now maximum number of nodes at the level l of a binary tree is 2 to the power l . Why is it so? Because at 0 you will have 1 node 2 to the power of 0 . Now this may have maximum of 2 children, so you will have 2 nodes which is 2 to the maximum of you may have less. At level 2 , we already have 2 nodes, so each can have 2 children, so it is 4 that is 2 to the power 2 and so on, so at level l the maximum that you can have is 2 to the power l number of nodes.

So given that if you look at a height h then the maximum number of, so you will have to, what is the total number of nodes n at the maximum. Let us say n , less than equal to the maximum is 1 at the root then 2 then 2 square and it goes on till you have the height h 2 to the power h . And this, as you know, this is a geometric progression, so this is 2 to the power h plus 1 minus 1 .

So, the maximum number of nodes is bounded by 2 to the power h plus 1 minus 1 . And also given the height the number of nodes has to be at least two, I mean, at least 1 more than that, because, that is $n-1$ edges, so the height would be longest if every node has a single child and therefore, it has to be $h+1$ or n will have to be greater than or equal to $h+1$.

So, given these two bounce, I can just do the algebra, write it differently, and say that h is maximum value of h can be $n-1$, which is a number of edges that you can have maximum and the minimum of it is if you just take the logarithm on this side you will say $\log$ of $n+1$ ceiling

because it could be a non-integer number -1 . And if I just look at the asymptotic approximation, then I will say that this is actually order $\log n$, and this is order n .

So, the main result we get is height will always be less than equal to order n , which is obvious, and less obvious is it will be greater than equal to. So, the minimum height will always be order $\log n$ or more. If I have a k -ary tree, this $\log$ as you know is taken with respect to 2 with 2 as base because we are dealing with binary trees. If the $\log$, if it is a k -ary tree it will be taken in terms of $\log k$.

(Refer Slide Time: 16:57)

The slide is titled "Non-linear Data Structures (9): Hash Table (Module 44)". It features a diagram of a hash table with three columns: "keys", "hash function", and "buckets". The "keys" column lists "John Smith", "Lisa Smith", and "Sandra Dee". The "hash function" column shows arrows pointing from each key to a specific bucket index (01, 02, and 14 respectively). The "buckets" column shows the corresponding values: "521-8976" for index 01, "521-1234" for index 02, and "521-9855" for index 14. Below the diagram, there are two bulleted lists. The first list, "A hash table may be using:", includes "Static or Dynamic Schemes", "Open Addressing", "2-Choice Hashing", and "and so on". The second list, "Examples of a hash table include:", includes "Associative arrays", "Database indexing", "Caches", and "and so on". A small video inset of a man is visible in the bottom right corner of the slide.

- **Hash Table (Hash Map):** implements an associative array abstract data type, a structure that can map keys to values by using a hash function to compute an index (hash code), into an array of buckets or slots, from which the desired value can be found

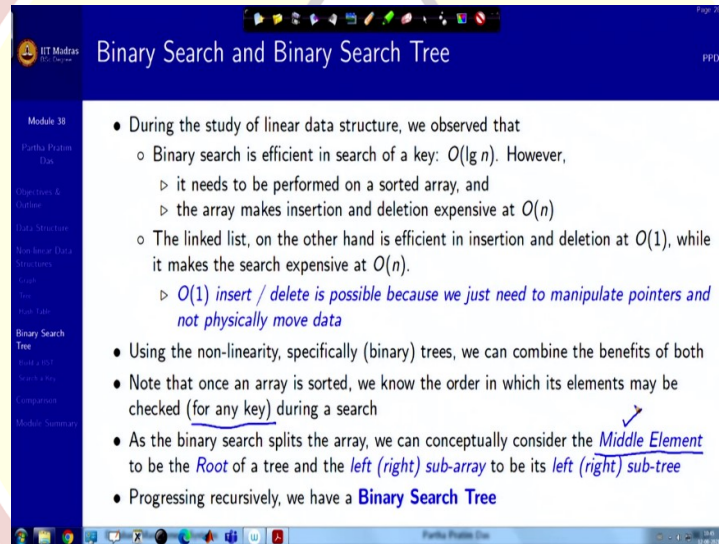
keys	hash function	buckets
John Smith	01	521-8976
Lisa Smith	02	521-1234
Sandra Dee	14	521-9855

- A hash table may be using:
 - Static or Dynamic Schemes
 - Open Addressing
 - 2-Choice Hashing
 - and so on
- Examples of a hash table include:
 - Associative arrays
 - Database indexing
 - Caches
 - and so on

Now, hash table is another concept where basically when you have to store the keys, you use a mathematical function to generate a number, typically an integer number, and use an array to store that particular value in that number. So, in this way, it becomes much easier to find a lot of information.

I will not go into depth explaining hash table here, this is just as a placeholder. Because in module 44, later, we are going to discuss in depth about hashing and its different applications.

(Refer Slide Time: 17:31)



Now, let me move on to the main thing that we want to discuss here is the binary search tree. Now, again from the study of the linear data structure, we found certain observations that binary search is one of the best possible way to find a key that was of $\log n$, but it has to be performed on a sorted array. And the moment you have array you have a deletion insertion, which is order n , so that prevented us from getting this benefit of finding the key efficiently.

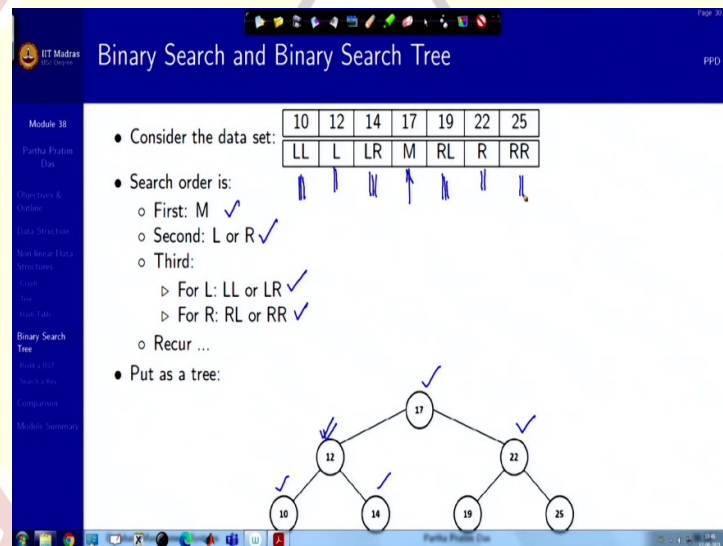
On the linked list on the other hand insertion deletion is order 1, but the search becomes expensive. And in the linked list, just to remind the insertion deletion can be order 1 because you do not need to move the data. In array you need to move the data, in linked list you just

move the pointers just references, so you can do that in a number of constant reference movements.

So, we want to use the non-linearity to combine the benefits of both these. So how do we do that? Let us say, I mean just conceptually think that data is an array and it is sorted. Once it is sorted, then for any key during a search, we know the order in which the elements may be checked. For example, we first check the middle element of the array. So, the middle element splits it into left sub array and right sub array.

What we can conceptualize that the entire array is a tree rooted in the middle element, and the left sub-tree is a left, left sub array is a left sub-tree and right sub array is a right sub-tree. And if we progress recursively we have a binary search tree.

(Refer Slide Time: 19:19)



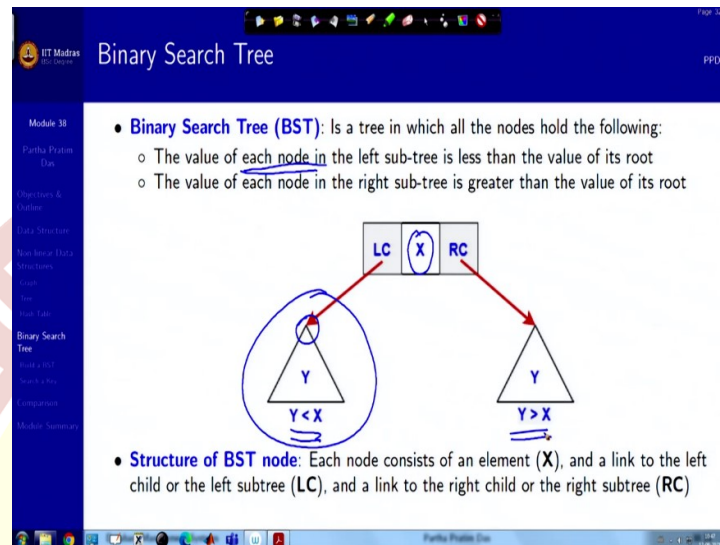
So, I will just illustrate through an example, consider a data set having seven data elements. I have purposefully chosen seven not to get into fractions. So, they are sorted here, and this is the middle element. So, no matter what is your key this is where you will do the first search, and I put that as a root here of the tree.

Then these form my left sub array, these form my right sub array. Left sub array, right sub array, left sub-tree, right sub-tree. Now once I have done the first then I will either decide to compare with L, the element at L or at R, no one else. So, these are the two sub roots, roots of the left sub-tree and the right sub-tree so the second will be L or R.

Then what is next? If I did L, if I did L, then I will either look at LL or I look at LR either this or this, the two other sub, sub arrays and sub-sub trees or if I had done R, I will look at either

RL or RR. So, the way we do structured binary search necessarily written out in terms of a tree organizes in this way, so that we get, and we just need to keep on recurring, so what we get as a result is what is formally defined as a binary search tree.

(Refer Slide Time: 20:48)



So, what is a binary search tree? Where it has a node representing, every node has a value, and possibly up to two children. So, I have a value X here in the root such that its left sub-tree all values in the left sub-tree is less than that each node. Please note this point. I have often seen in interviews that people forget this each node part, they just say, that the root of the left sub-tree has to be less than X that will be completely wrong, as you have seen, because the whole left sub-array is less than the middle element.

So, each node in this all Y will have to be less than X. And all on the right sub-tree all Y will have to be greater than that. If that satisfied at every node then we have a binary search tree exactly as we constructed from the binary search.

So, let us say, let us keep on inserting now to build this BST. Let us not pun intended, but let me insert an elephant. So E is the first. Now L, why should L go? L is greater than E, so it must occur on the right sub array or the right sub-tree, so I add it as the right child. Then this P. Where should it go?

So, what I do is, every time I have to. Suppose I have to search for P, I have to insert P. I check for P in the array, and where it is expected I will insert it. So, P is greater than E comes on the right side greater than L comes on the right side, so it becomes a right child of L. Then I have H. I search with E it is greater, I search with, I compare with L, so I compare with E it is greater, so I compare with L it is lower, so it is on the left side it does not exist I add it here. Then I add A, it is smaller than E, so I add it as a left child. So, it just goes on. I mean, it is as simple as that.

(Refer Slide Time: 22:54)

Binary Search Tree (3)

- **Example:** Obtain the BST by inserting the following values- E, L, P, H, A, N, T.

ELPHANT

ELPHANT

Unbalanced

Similarly, then I add N, because I, n is greater than E, I go to the right, greater than N, L go to the right, less than P go to the left, do it here, and finally T, which is greater than all of E, L, P and this is I have the binary search tree corresponding to E,L,P,H,A,N,T. Now, please note that this is not unique. You can see one very major difference in than the earlier one is, in the earlier one the left sub-tree and the right sub-tree had the same number of nodes.

Well, that is because I chose 7. But even without that they are, they can always be made compatible. Whereas here, the left sub-tree has only 1, whereas, the right sub-tree has 5, and this is a condition which we will say is a condition of being unbalanced, which is not a good thing to have. We will see that.

(Refer Slide Time: 23:56)

Searching a key in BST

```
search(root, key)
1. Compare the key with the element at root.
  1.1. If the key is equal to root's element then
    1.1.1. Element found and return
  1.2. else if the key is lesser than the root's element
    1.2.1 search(root.lc) #search on the left subtree
  1.3. else: #if the key is greater than the root's element
    1.3.1 search(root.rc) #search on the right subtree
```

Q

E < N

L < N

P > N

N = N

Page 38

Searching a key in BST

Module 18

Partha Pratim Das

Search & Sort

Search a Key

```

search(root, key)
1. Compare the key with the element at root.
1.1. If the key is equal to root's element then
1.1.1. Element found and return
1.2. else if the key is lesser than the root's element
1.2.1. search(root.lc) #search on the left subtree
1.3. else: #if the key is greater than the root's element
1.3.1. search(root.rc) #search on the right subtree

```

Now how do you search? As I said, it is very simple. You start with the root, compare. If it is equal, you say you have found the key. If it is less, you go to the left child. If it is more, go to the right child, and again, do that same thing.

So, to look for H, I am sorry. So, to look for H, you compare with E, it is greater. You go to the right child compare with L, it is less, you go to the left child compare with H it is equal, so you say I have got this node return that node, return that pointer.

If you are say, looking for Q, you will. Let us say you are looking for Q, greater than E you come to L, greater than L you come to P, greater than P you come to T, less than T there is no child, so you declare that Q is not there in your data structure, the search fails. So, that is simply how the things go.

So, now, the question is, how many given a key given any arbitrary key say, how many comparisons and node traversals would you need? What is your, the extreme that can happen? For example, if you look at A can be found in two comparisons, H can be found in three comparisons, L can be found in two comparisons, P can be found in three comparisons. So, you can always say that, it is basically the level of the node plus 1, because that path you will have to traverse.

So, what will be the worst? The worst will be, when you do not get it. So, anything say B, C, D will fail after two comparisons, because they will be expected after this A. Then anything which you say, if I have F, where it will be expected here. If I have G it will be expected here. If I have I, it will be expected here, if I have J, it will be expected here and so on, these all will need 3.

But when it did Q, it was expected at this point. So, it is 1, 2, 3, 4 level is 4, so level is 3, so, you need 4 comparisons. So, basically the maximum that you can get is a maximum of this expression, which is basically $h+1$ comparisons, that you can need. When the key does not exist and is expected, as a child of the leaf node, which has the longest path from the root.

Two things, one is, it is not there, because if it is there, then it will require some less, you can always have something which is not there. And what will maximize the level? If you have the longest path from the root. And what is the longest path from the root? Length of the longest path from the root is the height of the tree, so, you will require $h+1$.

So, we can say that you in general require order h comparisons. So, if h is less, the tree is good, if h is more the tree is bad, as simple as that. And we have already noted that h has bounce.

(Refer Slide Time: 27:52)

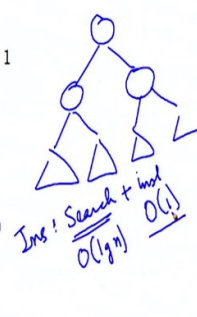
Searching a key in BST (2)

- Searching a key in a BST is $O(h)$, where h is the height of the key
- **Worst Case**
 - The BST is a skewed binary search tree (all the nodes except the leaf would have only one child)
 - This can happen if keys are inserted in sorted order
 - Height (h) of the BST having n elements becomes $n - 1$
 - Time complexity of search in BST becomes $O(n)$
- **Best Case**
 - The BST is a balanced binary search tree
 - This is possible if
 - ▷ If keys are inserted in purely randomized order, Or
 - ▷ If the tree is explicitly balanced after every insertion
 - Height (h) of the binary search tree becomes $\lg n$
 - Time complexity of search in BST becomes $O(\lg n)$

The diagram on the right shows a skewed binary search tree with a root node and four child nodes in a single chain, representing the worst case where the height is $n-1$.

Searching a key in BST (2)

- Searching a key in a BST is $O(h)$, where h is the height of the key
- Worst Case**
 - The BST is a skewed binary search tree (all the nodes except the leaf would have only one child)
 - This can happen if keys are inserted in sorted order
 - Height (h) of the BST having n elements becomes $n - 1$
 - Time complexity of search in BST becomes $O(n)$
- Best Case**
 - The BST is a balanced binary search tree
 - This is possible if
 - If keys are inserted in purely randomized order, Or
 - If the tree is explicitly balanced after every insertion
 - Height (h) of the binary search tree becomes $\lg n$
 - Time complexity of search in BST becomes $O(\lg n)$



So, given this, the searching of a key in the binary search tree is order H , the height of the key. The worst case will happen when all our nodes are like this, so that your height itself is $n-1$. So, necessarily your search time would become worst in the order n . And what is the good, the best that you can have.

When? At every node you have sub-trees, which are of the same size like you did in the binary search, the sub-trees are always of the same size. So, if that happens then the height, the bound and the height is $\log n$. So, if the bound and the height is $\log n$, all keys can be found in $\log n$.

And with the added advantage that all that you needed to do for insert or delete. So, what happens on insert? You do a search plus insert. You first find out where it should be, if it is not there, then you insert. Now search is basically order $\log n$, and this actual insertion is manipulating pointers, which is order 1, same thing will happen with delete.

So, it is not only that with this if I can keep the sub-trees of almost the same size then at every node that is what is balancing. Then the binary search tree could give me the search, the insert, the delete, everything in order $\log n$ time, which is much better than which I could do in any of the linear data structure. So that is the basic advantage.

(Refer Slide Time: 29:52)

IIT Madras
B.Tech Degree

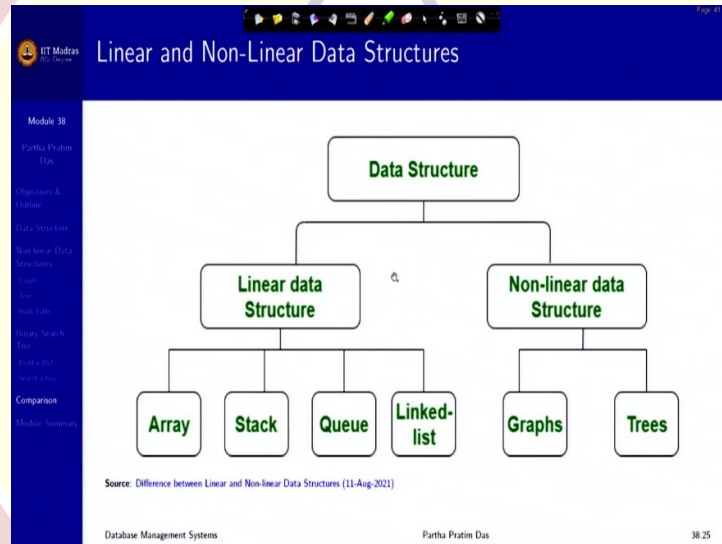
Comparison of Linear and Non-Linear Data Structures

Module 38
Partha Pratim Das

Objectives & Outline
Data Structure
Non-linear Data Structures
Graph
Tree
Hash Table
Binary Search Tree
Heap & B+T
Search & Key
Comparison
Module Summary

Comparison of Linear and Non-Linear Data Structures

Database Management Systems Partha Pratim Das 38/24



IIT Madras
B.Tech Degree

Comparison of Linear and Non-Linear Data Structures

Module 38
Partha Pratim Das

Objectives & Outline
Data Structure
Non-linear Data Structures
Graph
Tree
Hash Table
Binary Search Tree
Heap & B+T
Search & Key
Comparison
Module Summary

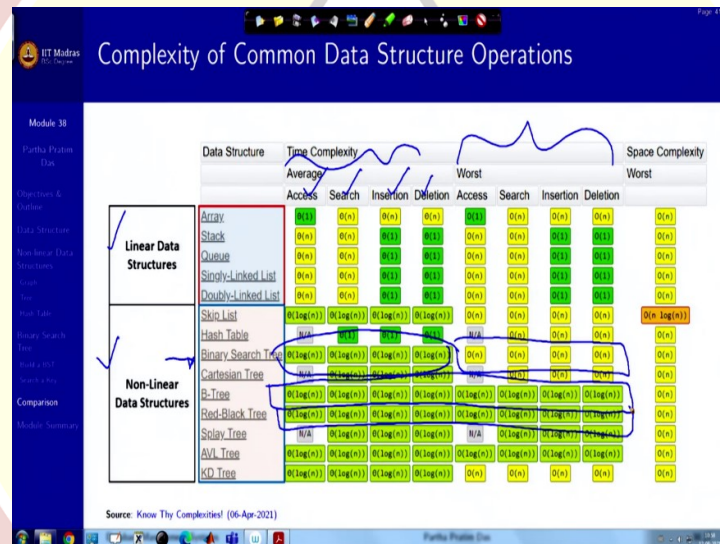
Linear Data Structure	Non-Linear Data Structure
• Data elements are <i>arranged</i> in a linear order where each and every elements are attached to its previous and next adjacent	• Data elements are <i>arranged</i> in a hierarchical or networked manner
• Single <i>level</i> is involved	• Multiple <i>level</i> are involved
• <i>Implementation</i> is easy in comparison to non-linear data structure	• <i>Implementation</i> is complex in comparison to linear data structure
• Data elements can be <i>traversed</i> in one way only	• Data elements can be <i>traversed</i> in multiple ways. Various traversal techniques are defined to linearize the data: Breadth-First, Inorder, Preorder, etc.

Database Management Systems Partha Pratim Das 38/26

Now, if you broadly compare this is the schematic of what we have discussed in this previous module, and this module, and have given a just pointwise comparison that linear has a direct arrangement, whereas, non-linear has hierarchical arrangement. Linear naturally has a single level, non-linear has multiple level. Linear is easier, non-linear is more difficult to implement.

Data elements in linear can be traversed in one way, whereas, data elements in a non-linear structure will can be traversed in multiple ways to linearize it. You can do a Depth-First traversal, Breadth-First traversal, in-order, post order, pre-order and so on. If you are not familiar, please look up your book on the algorithms and get familiar we are going to need all of them and the examples.

(Refer Slide Time: 30:46)



The chart displays the time and space complexities for various data structures. It is divided into Linear Data Structures and Non-Linear Data Structures. The operations measured are Access, Search, Insertion, and Deletion, each with Average and Worst case complexities. Space complexity is also shown for the Worst case. The chart uses a color-coded system: Green for the best performance, Yellow for intermediate, and Yellowish Green for the worst performance.

Data Structure	Time Complexity				Space Complexity			
	Average Access	Average Search	Average Insertion	Average Deletion	Worst Access	Worst Search	Worst Insertion	Worst Deletion
Linear Data Structures								
Array	$O(1)$	$O(n)$	$O(n)$	$O(n)$	$O(1)$	$O(n)$	$O(n)$	$O(n)$
Stack	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Queue	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Singly-Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Doubly-Linked List	$O(n)$	$O(n)$	$O(1)$	$O(1)$	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Non-Linear Data Structures								
Skipped List	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Hash Table	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Binary Search Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Cartesian Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
B-Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Red-Black Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
Splay Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
AVL Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$
KD Tree	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(\log(n))$	$O(1)$	$O(1)$	$O(1)$	$O(1)$

Source: Know Thy Complexities! (06-Apr-2021)

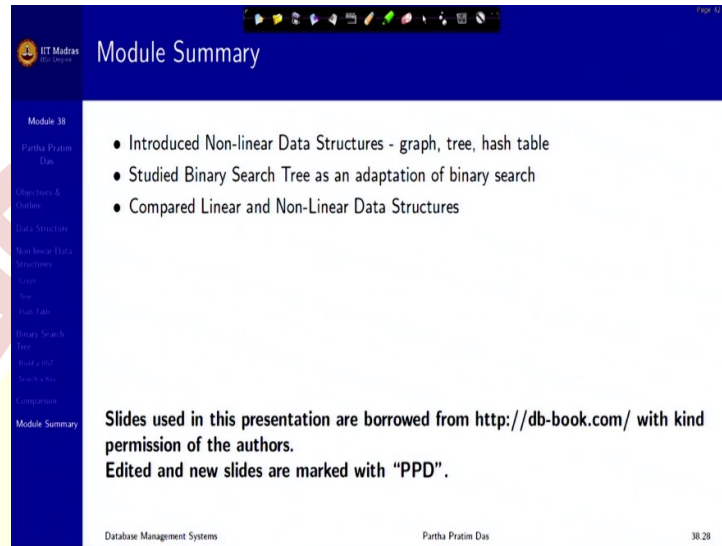
So, with this I conclude with a comparative chart of complexities of different operations access, search, insertion and deletion. In the average case, as well as in the worst case you can see that the colors mean, absolutely the best, yellow means kind of the worst, and this yellowish green means something in between, which we are trying to look at.

So, here when we were in the linear, we do not have any data structure which has only green or only your greenish yellow. So, that was a basic contention. But if you look at binary search tree, we have all greenish yellow in the average case, the worst case is still bad. So, we will have to make sure that we do not hit the worst case.

But there are trees, there are nonlinear data structures that we are going to discuss later on where like Red Black Tree like B-Tree these I am going to discuss in subsequent modules,

where you will see that no matter whether it is a average case, or worst case, no matter which operation you are doing everything is order $\log n$, and that is the best that we can get. And the good thing is there is no space overrated for most of them, all of them use the order n number of spaces.

(Refer Slide Time: 32:08)



Module Summary

- Introduced Non-linear Data Structures - graph, tree, hash table
- Studied Binary Search Tree as an adaptation of binary search
- Compared Linear and Non-Linear Data Structures

Slides used in this presentation are borrowed from <http://db-book.com/> with kind permission of the authors.
Edited and new slides are marked with "PPD".

Database Management Systems | Partha Pratim Das | 38/28

So, with that, I conclude this module. Thank you very much for your attention. And we will meet in the next module.